

베타테스트 자동화 도구 기술 스택 명세서

Technology Stack Specification

구현 계획서 v2.0 / 아키텍처 상세 설계서를 참조한
컴포넌트별 기술 선정·버전·대안 비교

작성일: 2026 년 5 월 19 일 | 버전: 1.0

0. 문서 개요

0.1 목적

이 문서는 베타테스트 자동화 도구의 모든 컴포넌트에 대해 채택한 기술 스택을 정의한다. 각 기술은 (1) 선정 사유, (2) 검토한 대안, (3) 버전 정책, (4) 향후 확장 시 대체 가능성을 함께 명시한다.

참조 문서:

- 구현 계획서 v2.0 — 무엇을 언제 만들지
- 시스템 아키텍처 상세 설계서 — 어떻게 구조화할지
- 본 문서 — 무엇으로 구현할지

0.2 기술 선정 원칙

원칙 1. Python 단일 언어 우선 (Backend)

Backend 는 Python 으로 통일한다. LLM, GitHub, Google Sheets 모두 Python SDK 가 가장 성숙하고, 1 인 개발자가 컨텍스트 전환 비용을 최소화할 수 있다.

원칙 2. 성숙도 우선

새로운 기술보다 충분히 검증된 기술을 우선 채택한다. 새로운 도구(uv, ruff 등)도 도입하지만, 이는 이미 업계 표준에 가깝게 자리잡은 것들에 한한다.

원칙 3. 교체 가능성

외부 시스템과의 연동은 모두 ABC(Abstract Base Class)로 추상화한다. LLM Provider, Sheet 백엔드 등 교체 가능성이 있는 모든 기술은 인터페이스 뒤에 둔다.

원칙 4. 최소 의존성

같은 기능을 제공하는 두 라이브러리가 있을 때, 이미 다른 곳에서 쓰이는 것을 선택한다. 의존성 그래프를 알게 유지한다.

원칙 5. 확장 가능성

MVP 단계에서는 단순하게 시작하되, Phase 2~4 확장 시 자연스럽게 진화할 수 있는 기술을 선택한다.

0.3 문서 구성

- 1 장: 전체 기술 스택 요약 — 4 계층 + 횡단 관심사 한눈에 보기
- 2 장: Frontend 스택 — Next.js 기반 웹 폼
- 3 장: Backend 스택 — FastAPI + 서비스 레이어
- 4 장: 데이터 및 인프라 스택 — DB, Secret, 배포, 모니터링
- 5 장: 주요 기술 선택의 대안 비교 — 채택/미채택 결정
- 6 장: 의존성 관리 및 버전 정책
- 7 장: 확장 시 추가/교체될 기술
- 부록: ADR 매핑, 라이선스 점검

1. 전체 기술 스택 요약

아키텍처 설계서의 4 계층 + 횡단 관심사를 기준으로 채택 기술을 정리한다.

계층 / 영역	채택 기술	권장 버전	역할
Frontend	Next.js + TypeScript	Next 15.x, TS 5.6+	웹 폼 UI, SSR/SSG
Frontend	Tailwind CSS	4.x	스타일링
Frontend	React Hook Form + Zod	RHF 7.x, Zod 3.x	폼 상태 + 클라이언트 검증
API Layer	FastAPI	0.115+	HTTP 엔드포인트, OpenAPI 자동 생성
API Layer	uvicorn (ASGI)	0.32+	FastAPI ASGI 서버
API Layer	Pydantic v2	2.9+	요청 검증, DTO 정의
Service Layer	Python 표준 + abc	Python 3.12+	비즈니스 로직, ABC 인터페이스
Service Layer	dependency-injector	4.x	의존성 주입 컨테이너 (선택)
Adapter — GitHub	PyGithub + httpx	PyGithub 2.5+, httpx 0.27+	Issue/PR 생성, Webhook 수신
Adapter — LLM	anthropic SDK	0.40+	Claude API 호출, tool use
Adapter — Sheet	google-api-python-client	2.150+	Sheets API v4 호출
Adapter — Slack	httpx (직접 호출)	0.27+	Incoming Webhook POST
Data — 영구	GitHub Issues + Google Sheets	-	외부 시스템이 데이터 보관
Data — 로컬 (W4)	SQLite	3.40+	이벤트 로그, 행 인덱스 캐시
Data — ORM (W4)	SQLModel	0.0.22+	FastAPI 친화 ORM

계층 / 영역	채택 기술	권장 버전	역할
배포 — Backend	Google Cloud Run + Docker	-	자동 스케일링 컨테이너
배포 — Frontend	Vercel	-	Next.js 최적화 호스팅
스케줄러	GitHub Actions cron	-	매일 17 시 trigger
Secret 관리	Google Secret Manager	-	키·자격 증명 보관
로깅	structlog + Cloud Logging	structlog 24+	구조화된 JSON 로그
에러 추적	Sentry (Phase 2)	최신	프로덕션 예외 자동 알림
패키지 관리 (Py)	uv	0.5+	의존성 해결, lock 파일
패키지 관리 (JS)	pnpm	9.x+	workspaces 지원
Lint / Format	ruff + mypy + ESLint + Prettier	ruff 0.7+, mypy 1.13+	코드 품질
테스트	pytest + pytest-asyncio + httpx 테스트	pytest 8+	단위·통합 테스트
CI/CD	GitHub Actions	-	lint → test → build → deploy

2. Frontend 스택

Frontend 는 베타 테스터가 직접 접하는 유일한 컴포넌트다. 사내 SSO 뒤에서 동작하며, Backend API 만 호출한다.

2.1 Next.js 15 (App Router)

선정 이유	React 기반 표준 프레임워크 / Vercel 호스팅과 완벽 통합 / App Router 로 폼 페이지 분리 깔끔 / SSO 미들웨어를 Edge Middleware 로 쉽게 구현 가능
버전 정책	15.x 메이저 핀, 마이너는 자동 업데이트. App Router 안정화 이후 버전 사용
검토한 대안	Vite + React: SPA 단순하지만 SSR/Edge Middleware 부재로 SSO 처리 복잡 / SvelteKit: 학습 비용 / Remix: Next.js 대비 생태계 작음
Phase 2 변화	변화 없음. Frontend 는 안정적인 단일 앱으로 유지

2.2 TypeScript 5.6+

선정 이유	Backend Pydantic 모델과 형태 일치 검증 / IDE 자동 완성 / 폼 필드와 API 응답 타입 안정성
strict 모드	tsconfig.json 에 strict: true 명시. noImplicitAny, strictNullChecks 활성화

2.3 Tailwind CSS 4

선정 이유	빠른 프로토타이핑 / 1 인 개발자에게 디자인 시스템 일관성 확보 용이 / 별도 CSS 파일 관리 부담 없음
검토한 대안	CSS Modules: 클래스 충돌 없지만 디자인 토큰 직접 관리 / Styled-Components: 런타임 비용 / shadcn/ui: 좋지만 컴포넌트 수가 적어 오버킬

2.4 React Hook Form + Zod

선택 이유	이슈 폼 항목이 15+개로 많아 폼 상태 관리 필수 / Zod 스키마를 packages/shared-schemas 에 두어 Backend Pydantic 과 1:1 대응 / 리렌더 최소화
검토한 대안	Formik: 더 무거움 / TanStack Form: 신생, 안정성 미검증 / 기본 useState: 항목이 많아 비효율

2.5 Frontend 보조 라이브러리

라이브러리	버전	용도
@hookform/resolvers	3.x	RHF 와 Zod 통합
lucide-react	0.460+	아이콘 (트리 셰이킹 가능)
date-fns	4.x	날짜 포매팅 (KST)
react-dropzone	14.x	이미지 드래그&드롭 업로드
sonner	1.x	토스트 알림 (제출 결과)

3. Backend 스택

Backend 는 단일 FastAPI 애플리케이션이지만 4 개 레이어로 모듈화되어 있다. 각 레이어의 채택 기술을 정리한다.

3.1 API Layer

3.1.1 FastAPI 0.115+

선정 이유	타입 안전성: Pydantic 통합으로 요청·응답 자동 검증 / OpenAPI 자동 생성: 팀 합류 시 API 문서 자동 / 비동기 지원: httpx, anthropic SDK 비동기 호출 가능 / BackgroundTasks 내장: Webhook 처리에 활용 / 의존성 주입 시스템: DI 컨테이너 별도 도입 불필요
검토한 대안	Flask: 비동기 지원 약함, 검증 수동 / Django: 무거움, ORM 불필요 / Starlette: FastAPI 의 기반이지만 검증 직접 / Litestar: 신생, 생태계 작음
Phase 변화	Phase 2: Worker 분리 시에도 FastAPI 유지 (Worker 도 같은 코드베이스). Phase 3: API 와 Worker 가 별도 Cloud Run 서비스로 분리

3.1.2 uvicorn 0.32+ (ASGI Server)

FastAPI 의 표준 ASGI 서버. 프로덕션에서는 워커 수를 Cloud Run instance 메모리에 맞춰 조정 (보통 1~2 워커).

- Cloud Run 환경: 단일 워커 (각 instance 는 동시에 80 개 요청 처리)
- --workers 옵션 대신 --reload (개발) 또는 단일 프로세스 (프로덕션)

3.1.3 Pydantic v2 (2.9+)

선정 이유	FastAPI 기본 통합 / v2 는 Rust 코어로 v1 대비 5~50 배 빠름 / JSON Schema 자동 생성 → packages/shared-schemas 에서 TS Zod 와 동기화
주요 활용	BugReport, EnhancementRequest 모델 / 환경변수 → pydantic-settings / LLMAnalysisResult 응답 검증

3.2 Service Layer

Service Layer 는 외부 의존성 없이 Python 표준 + abc 만 사용한다. Adapter ABC 를 통해 외부 시스템과 통신한다.

3.2.1 abc (Python 표준)

LLMPort, SheetPort, GitHubPort, NotifierPort 등 모든 Adapter 인터페이스를 ABC 로 정의. Service 는 ABC 만 의존하고 구현체는 의존성 주입으로 받는다.

```
from abc import ABC, abstractmethod

class LLMPort(ABC):
    @abstractmethod
    async def analyze(self, ...) -> LLMAnalysisResult: ...
```

3.2.2 의존성 주입 — 두 가지 선택지

의존성 주입은 두 방식 중 하나로 처리한다. MVP 는 FastAPI 내장 사용을 권장.

방식	시점	설명
FastAPI Depends	MVP (W1)	FastAPI 의 Depends() 시스템 사용. 의존성을 함수 시그니처에 명시. 가벼움
dependency-injector	Phase 2	Service Layer 가 커지고 의존성 그래프 복잡해질 때 도입. Container 패턴으로 명시적 관리

3.3 Adapter Layer

3.3.1 PyGithub 2.5+ + httpx (GitHubAdapter)

선택 이유	PyGithub 는 객체 모델로 직관적 / GitHub App 인증 지원 / 다만 일부 API 는 미지원이라 httpx 로 직접 호출 보강
주요 사용	Issue/PR 생성, 라벨 부여, 브랜치/커밋 (PyGithub) / Tree API, 일부 신규 GraphQL (httpx)
검토한 대안	gidgethub: 비동기 친화적이지만 객체 모델 없음 / githubkit: 신생

3.3.2 anthropic SDK 0.40+ (LLMAdapter)

선정 이유	Claude API 공식 SDK / 비동기 호출 지원 / 자동 재시도·타임아웃 내장 / Tool Use 지원 (향후 구조화 응답 강화 가능)
모델 선택	claude-sonnet-4-6: 비용/품질 균형 (MVP 기본) / claude-opus-4-7: 복잡한 패치 생성 시 (옵션)
프롬프트 형식	system: 역할 + 출력 스키마 지정 / user: 이슈 본문 + 코드 컨텍스트 / response_format: JSON 강제
Phase 변화	LLMPort ABC 덕분에 OpenAI, 자체 모델 등으로 교체 가능. AnthropicAdapter, OpenAIAdapter 병존 가능

3.3.3 google-api-python-client 2.150+ (SheetAdapter)

선정 이유	공식 Google SDK / batchUpdate, valueRenderOption 등 전체 Sheets API 사용 가능 / 서비스 계정 인증 표준
gsread 비교	gsread 는 단순하지만 batchUpdate 지원 제한 / 본 시스템은 API 호출 횟수가 많아 batch 필수 → 공식 SDK 채택
주요 사용	values().append, values().batchUpdate, spreadsheets().batchUpdate (조건부 서식 등)

3.3.4 httpx 0.27+ (공통 HTTP 클라이언트)

Slack Webhook, SSO IdP, GitHub 의 일부 API 를 httpx 로 호출한다. requests 대신 httpx 인 이유:

- 동기/비동기 모두 지원 (async def 함수에서도 자연스럽게)
- HTTP/2 지원
- Connection pooling 자동
- FastAPI 테스트에서 표준으로 사용 (TestClient 가 httpx 기반)

3.4 Backend 보조 라이브러리

라이브러리	버전	용도
pydantic-settings	2.6+	환경변수 → 설정 객체 변환, 타입 검증
structlog	24.4+	구조화된 JSON 로그 (trace_id 포함)

라이브러리	버전	용도
tenacity	9.x	지수백오프 재시도 (외부 API 호출에 적용)
python-multipart	0.0.17+	FastAPI 파일 업로드 (이미지 첨부)
PyJWT	2.9+	GitHub App JWT 생성, SSO 토큰 검증
cryptography	44+	GitHub Webhook HMAC 검증, PEM 파싱
orjson	3.10+	FastAPI 응답 JSON 직렬화 가속
python-frontmatter	1.1+	Issue Body 의 YAML 헤더 파싱

4. 데이터 및 인프라 스택

4.1 데이터 저장소

아키텍처 설계서 4.3 절의 데이터 보관 정책에 따라 다음과 같이 저장소를 사용한다.

데이터	저장소	선정 사유
Issue / PR	GitHub	단일 진실의 출처. 별도 DB 불필요
Sheet 행 / Snapshot	Google Sheets	운영자 가시성. API 로 read/write 가능
이벤트 로그	SQLite (W4 도입)	로컬 파일, 백업·복원 단순
행 인덱스 캐시	메모리 (W2) → SQLite (W4)	Issue 번호 → Sheet 행 매핑. 성능 향상
Secret	Google Secret Manager	키 회전, IAM 통합, Cloud Run 자동 마운트

4.2 SQLite + SQLAlchemy (W4 이후)

선정 이유	MVP 에서 PostgreSQL 은 과함 / SQLite 는 파일 1 개로 운영 단순 / Cloud Run 의 영구 볼륨(Cloud Storage FUSE) 또는 Litestream 으로 백업 가능
ORM 선택	SQLModel: Pydantic + SQLAlchemy 결합. FastAPI 친화. 같은 모델로 API·DB 동시 사용 가능
마이그레이션	Alembic 사용. 초기 스키마부터 버전 관리
Phase 3 변화	팀 합류 시 PostgreSQL (Cloud SQL)로 이전. SQLModel 은 그대로, 연결 문자열만 변경

4.3 배포 인프라

4.3.1 Google Cloud Run (Backend)

선정 이유	컨테이너 기반: Docker 그대로 / 자동 스케일링: 부하 없을 때 0 인스턴스 (비용 절감) / Secret Manager 통합 / 사내 VPC 연동 가능 / 무료 티어 충분 (월 200 만 요청)
설정	min instances: 1 (콜드 스타트 회피) / max: 10 / 메모리: 1GB / CPU: 1 / 동시 요청: 80
검토한 대안	AWS Lambda: 컨테이너 한계 (10GB 이미지), 콜드 스타트 / Fly.io: 좋지만 신뢰성 GCP 대비 약함 / GKE: 1 인 운영 부담 / Vercel Functions: Python 지원 약함

4.3.2 Vercel (Frontend)

선정 이유	Next.js 만든 회사 / Edge Middleware 로 사내 IP 제한 구현 / GitHub 연동으로 PR 마다 preview / 무료 티어 충분
Edge Middleware	미들웨어에서 사내 IP 화이트리스트 검사 + SSO 검증. 인증 안 된 요청은 사내 SSO 로 리다이렉트

4.3.3 Docker

Backend 는 Docker 컨테이너로 배포. 멀티스테이지 빌드로 이미지 크기 최소화.

- Base: python:3.12-slim (약 130MB)
- Builder stage: uv 로 의존성 설치
- Runtime stage: 의존성 + 코드만 복사
- 최종 이미지 크기 목표: 300MB 이하

4.4 스케줄러

4.4.1 GitHub Actions cron (MVP)

매일 17 시 KST 일일 동기화를 트리거한다. 별도 인프라 불필요. 워크플로우는 단순히 Backend 의 /admin/sync 엔드포인트를 호출.

```
on:
  schedule:
    - cron: '0 8 * * *' # UTC 08:00 = KST 17:00
```

참고. GitHub Actions cron 은 약간의 지연이 있을 수 있다 (최대 10 분). 정확한 시간 보장이 필요하다면 Cloud Scheduler 로 이전.

4.4.2 APScheduler (Phase 2 검토)

Backend 내부에 스케줄러를 내장하는 옵션. 다만 Cloud Run 이 idle 시 0 instance 가 되면 cron 이 동작하지 않으므로 min instances 를 유지해야 함. MVP 에서는 GitHub Actions 가 더 단순.

4.5 모니터링 및 로깅

4.5.1 structlog + Cloud Logging

모든 로그는 JSON 구조로 stdout 에 출력. Cloud Run 이 자동으로 Cloud Logging 에 수집.

```
logger.info(  
    'issue_created',  
    issue_number=142,  
    tester_id='id1234',  
    elapsed_ms=234,  
    trace_id=ctx.trace_id  
)
```

- Cloud Logging 은 7 일 무료 보관 (기본)
- 필터: severity, issue_number, trace_id 로 검색 가능
- Phase 2 이후 BigQuery 로 익스포트하여 장기 보관

4.5.2 Sentry (Phase 2)

프로덕션 예외 자동 캡처 및 Slack 알림. 무료 플랜으로 월 5,000 이벤트 가능. MVP 에서는 구조화 로그 + Slack 알림으로 대체 가능하므로 즉시 도입하지 않음.

4.5.3 Cloud Monitoring 알람

- 5xx 비율 > 5% → Slack 알림
- p95 응답시간 > 3 초 → Slack 알림
- LLM 일일 비용 > 한도의 80% → Slack 알림
- 동기화 실패 → Slack 즉시 알림 (Backend 가 직접 발송)

5. 주요 기술 선택의 대안 비교

이 장은 의사결정 추적성을 위해 작성한다. 각 선택지에 대해 검토한 대안과 채택/미채택 사유를 정리한다.

5.1 Backend Framework

후보	결정	장점	미채택 사유
FastAPI	채택	타입 안전성, OpenAPI 자동, 비동기, Pydantic 통합	-
Flask	미채택	단순, 학습 자료 풍부, 작은 앱에 적합	비동기 약함, 타입 검증 수동, OpenAPI 별도 도입
Django	미채택	ORM·관리자·인증 통합 풀스택	ORM 불필요 (외부 시스템이 데이터), 무거움
Litestar	미채택	FastAPI 대안, 성능 약간 우위	커뮤니티·학습 자료 작음, 1인 개발 위험

5.2 LLM Provider

후보	결정	장점	비고
Claude (Anthropic)	채택	코드 분석·diff 생성 품질 우수, Tool Use 안정적	LLMPort ABC로 추상화되어 교체 가능
OpenAI (GPT-4)	대안 보유	범용성, 생태계 큼	Phase 2+에 OpenAIAdapter 추가 가능
자체 호스팅 LLM	미채택	비용 절감, 데이터 외부 노출 없음	운영 부담 큼, 품질 미보장. Phase 4 검토

5.3 Backend 호스팅

후보	결정	장점	미채택 사유
Cloud Run	채택	컨테이너, 자동 스케일, Secret 통합	-

후보	결정	장점	미채택 사유
AWS Lambda	미채택	Serverless 표준	Python 컨테이너 한계, 콜드 스타트 더 큼
Fly.io	미채택	저렴, 가까운 리전 (한국)	GCP 대비 안정성·통합성 약함
GKE (Kubernetes)	미채택	최고 유연성·확장성	1 인 운영 부담 큼. Phase 4 검토

5.4 데이터 저장소 (이벤트 로그)

후보	결정	장점	비고
SQLite (MVP)	채택	파일 1 개, 백업 단순, 운영 부담 0	W4 도입
PostgreSQL	Phase 3 채택	동시성, 고급 쿼리, 운영 도구 풍부	팀 합류 시 Cloud SQL 로 이전
MongoDB	미채택	JSON 친화	관계형 데이터에 부적합, 운영 부담
Firestore	미채택	GCP 통합, NoSQL Serverless	쿼리 제약, 비용 예측 어려움

5.5 Message Queue (Phase 2 예정)

후보	결정	장점	비고
BackgroundTasks (MVP)	채택	FastAPI 내장, 의존성 없음	프로세스 재시작 시 작업 손실 (수용)
Redis + RQ	Phase 2 후보	단순, 운영 부담 적음	LLM 동시성 5+ 시 도입 검토
Celery + Redis	Phase 2 후보	기능 풍부, 재시도·스케줄링	복잡도 큼. RQ 먼저 검토
Cloud Tasks	미채택	GCP 통합, 관리형	벤더 락인, RQ 로 충분

6. 의존성 관리 및 버전 정책

6.1 Python — uv

uv 는 Astral 이 만든 차세대 Python 패키지 관리자. Rust 기반으로 pip/poetry 대비 10~100 배 빠르다.

- pyproject.toml 로 의존성 선언
- uv.lock 으로 정확한 버전 고정
- uv sync 단일 명령으로 가상환경 + 의존성 설치
- Docker 빌드에서 uv 를 사용하여 빌드 시간 단축

주요 의존성 (apps/api/pyproject.toml)

```
[project]
name = "sizl-beta-tool-api"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = [
    "fastapi>=0.115",
    "uvicorn[standard]>=0.32",
    "pydantic>=2.9",
    "pydantic-settings>=2.6",
    "httpx>=0.27",
    "anthropic>=0.40",
    "PyGithub>=2.5",
    "google-api-python-client>=2.150",
    "google-auth>=2.35",
    "structlog>=24.4",
    "tenacity>=9.0",
    "python-multipart>=0.0.17",
    "PyJWT[crypto]>=2.9",
    "orjson>=3.10",
    "python-frontmatter>=1.1",
]

[dependency-groups]
dev = [
    "pytest>=8.0",
    "pytest-asyncio>=0.24",
    "pytest-mock>=3.14",
    "ruff>=0.7",
    "mypy>=1.13",
]
```

6.2 JavaScript — pnpm workspaces

apps/web 과 packages/* 를 모노레포로 관리. pnpm 은 npm/yarn 대비 디스크 효율적이고 strict 하다.

- pnpm-workspace.yaml 로 워크스페이스 선언
- pnpm-lock.yaml 로 정확한 버전 고정
- Turborepo 는 일단 도입하지 않음 (앱이 2 개라 오버킬). 팀 합류 시 검토

6.3 버전 핀 정책

범주	핀 방식	예시
Framework (FastAPI, Next.js)	메이저 핀, 마이너 ~	fastapi>=0.115,<0.116
Adapter SDK (anthropic, github)	메이저 핀, 마이너 자유	anthropic>=0.40 (Breaking 시 메이저 업)
보안 관련 (cryptography, PyJWT)	최신 마이너 자동	Dependabot 으로 자동 PR
개발 도구 (ruff, mypy)	느슨 핀, 분기마다 업데이트	ruff>=0.7

6.4 보안 및 취약점 관리

- Dependabot 활성화: 보안 패치 자동 PR (GitHub 기능)
- CI 에 pip-audit 통합: 매 PR 마다 의존성 취약점 스캔
- npm audit (CI 에서): Frontend 의존성 스캔
- 월 1 회 수동 점검: 메이저 업데이트 검토 + 변경점 확인

6.5 라이선스 점검

CI 에 라이선스 체크 통합. 다음 라이선스만 허용:

- MIT, Apache 2.0, BSD-2/3-Clause, ISC: 자유 사용
- LGPL: 동적 링크 한해 사용
- GPL, AGPL: 검토 후 결정 (보통 회피)

참고. *FastAPI*(MIT), *Pydantic*(MIT), *anthropic SDK*(MIT), *PyGithub*(LGPL), *Next.js*(MIT) 모두 허용 라이선스.

7. 확장 시 추가 또는 교체될 기술

구현 계획서 v2.0 의 12 장에서 정의한 Phase 2~4 에 어떤 기술이 추가·교체되는지 정리한다.

7.1 Phase 2: LLM Worker 분리

기술	변화	사유
Redis	신규 추가	LLM 작업 큐. Memorystore for Redis 사용
RQ (Redis Queue)	신규 추가	Celery 보다 단순. RQ Scheduler 별도 도입
apps/llm-worker	신규 분리	기존 services/llm 을 별도 Cloud Run 서비스로
Sentry	신규 도입	프로덕션 예외 추적, 멀티 서비스 관리

7.2 Phase 3: 팀 합류 (3 인+)

기술	변화	사유
PostgreSQL (Cloud SQL)	SQLite 교체	동시성, 백업, 모니터링 도구 풍부
Alembic	계속 사용	DB 마이그레이션 (PostgreSQL 에서도 동일)
apps/sheet-sync	신규 분리	Sheet Syncer 를 별도 서비스로 분리
dependency-injector	신규 도입	Service Layer 의존성 그래프 명시적 관리
CODEOWNERS, PR 템플릿	신규 추가	팀 코드 리뷰 프로세스

7.3 Phase 4: 멀티 부서 확산

기술	변화	사유
GKE Autopilot	Cloud Run 교체 검토	멀티 서비스 운영 시 효율적

기술	변화	사유
Prometheus + Grafana	신규 도입	커스텀 메트릭, 대시보드
OpenTelemetry	신규 도입	분산 트레이싱 (멀티 서비스)
Tenant 추상화	코드 변경	멀티 저장소·멀티 부서 지원

8. 부록

8.1 ADR 매핑

기술 선택 중 의사결정 로그(ADR)로 별도 문서화할 항목.

ADR #	주제	결정 요약
ADR-001	Backend 언어/프레임워크	Python 3.12 + FastAPI 채택
ADR-002	Frontend 프레임워크	Next.js 15 (App Router) 채택
ADR-003	LLM Provider	Anthropic Claude 채택. LLMPort 로 추상화
ADR-004	Backend 호스팅	Google Cloud Run 채택
ADR-005	초기 DB 전략	MVP 에는 DB 없음 → W4 에 SQLite → Phase 3 에 PostgreSQL
ADR-006	Message Queue 도입 시점	MVP 는 BackgroundTasks → Phase 2 에 Redis + RQ
ADR-007	패키지 관리자	Python 은 uv, JS 는 pnpm 채택
ADR-008	스케줄러 위치	GitHub Actions cron (외부). Backend 내장 안 함

8.2 라이선스 점검 결과

주요 의존성의 라이선스를 정리한다. 모두 허용 가능한 오픈소스.

라이브러리	라이선스	비고
FastAPI	MIT	허용
Pydantic	MIT	허용
httpx	BSD-3-Clause	허용
anthropic SDK	MIT	허용
PyGithub	LGPL-3.0	동적 링크만 사용 — 허용
google-api-python-client	Apache-2.0	허용

라이브러리	라이선스	비고
structlog	MIT/Apache-2.0	허용
Next.js	MIT	허용
Tailwind CSS	MIT	허용
React Hook Form	MIT	허용
Zod	MIT	허용

8.3 관련 문서

- 구현 계획서 v2.0: 무엇을 언제 만들지
- 시스템 아키텍처 상세 설계서 v1.0: 어떻게 구조화할지
- 본 문서: 무엇으로 구현할지
- (향후 작성) PRD: 누구를 위해, 왜 만들지

8.4 변경 이력

- v1.0 (2026-05-19): 초안 작성. 구현 계획서 v2.0 의 기술 스택을 상세화

8.5 향후 보완 항목

- LLM 비용 측정 후 모델 선택 가이드 (Sonnet vs Opus)
- 부하 테스트 결과에 따른 Cloud Run 스펙 조정
- Phase 2 진입 시점에 Redis vs Cloud Tasks 재평가
- Frontend 인증을 Vercel Edge Middleware 로 구현할지, NextAuth 로 할지 (W1 결정)